

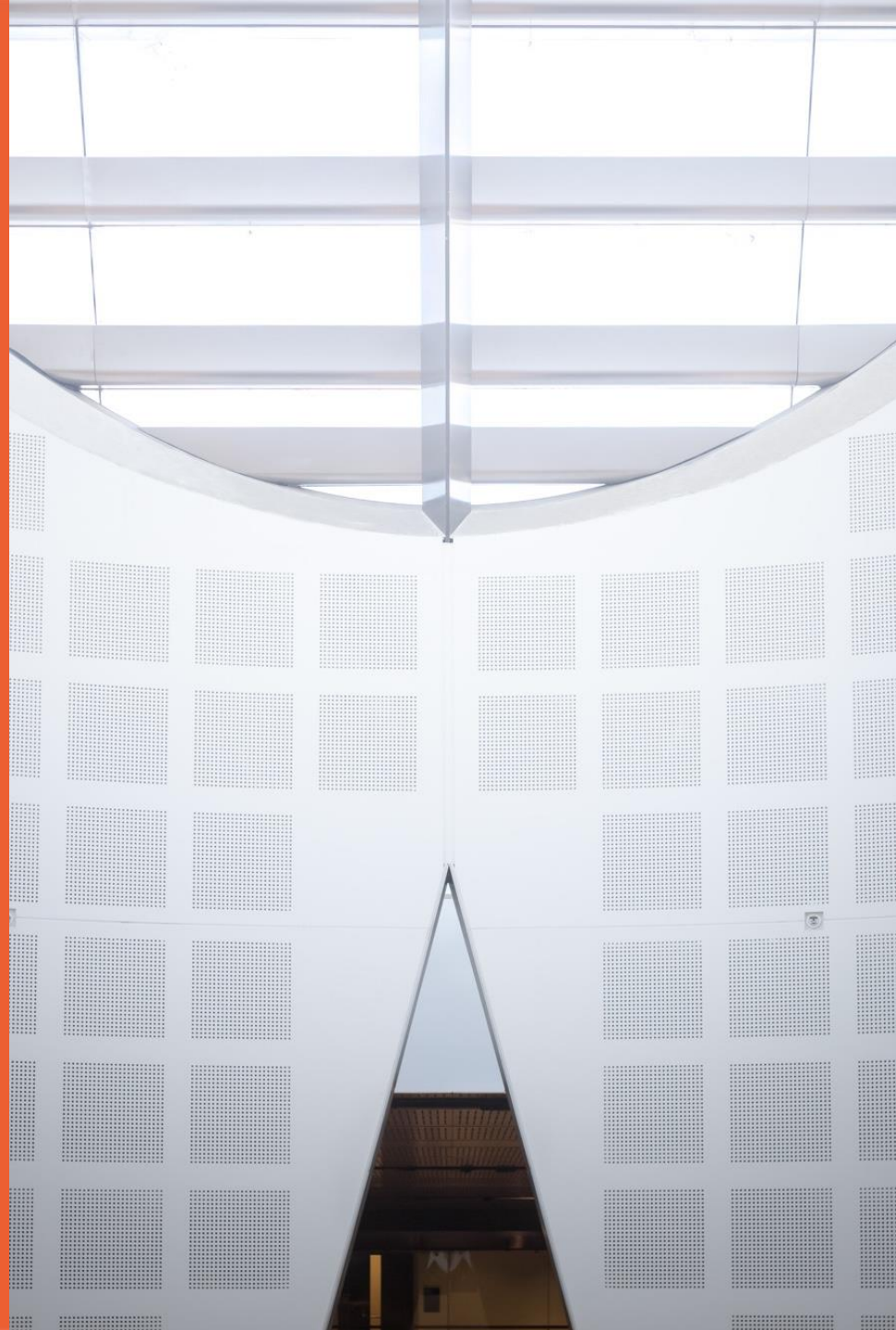
Deep Reinforcement Learning

A/Prof Chang Xu

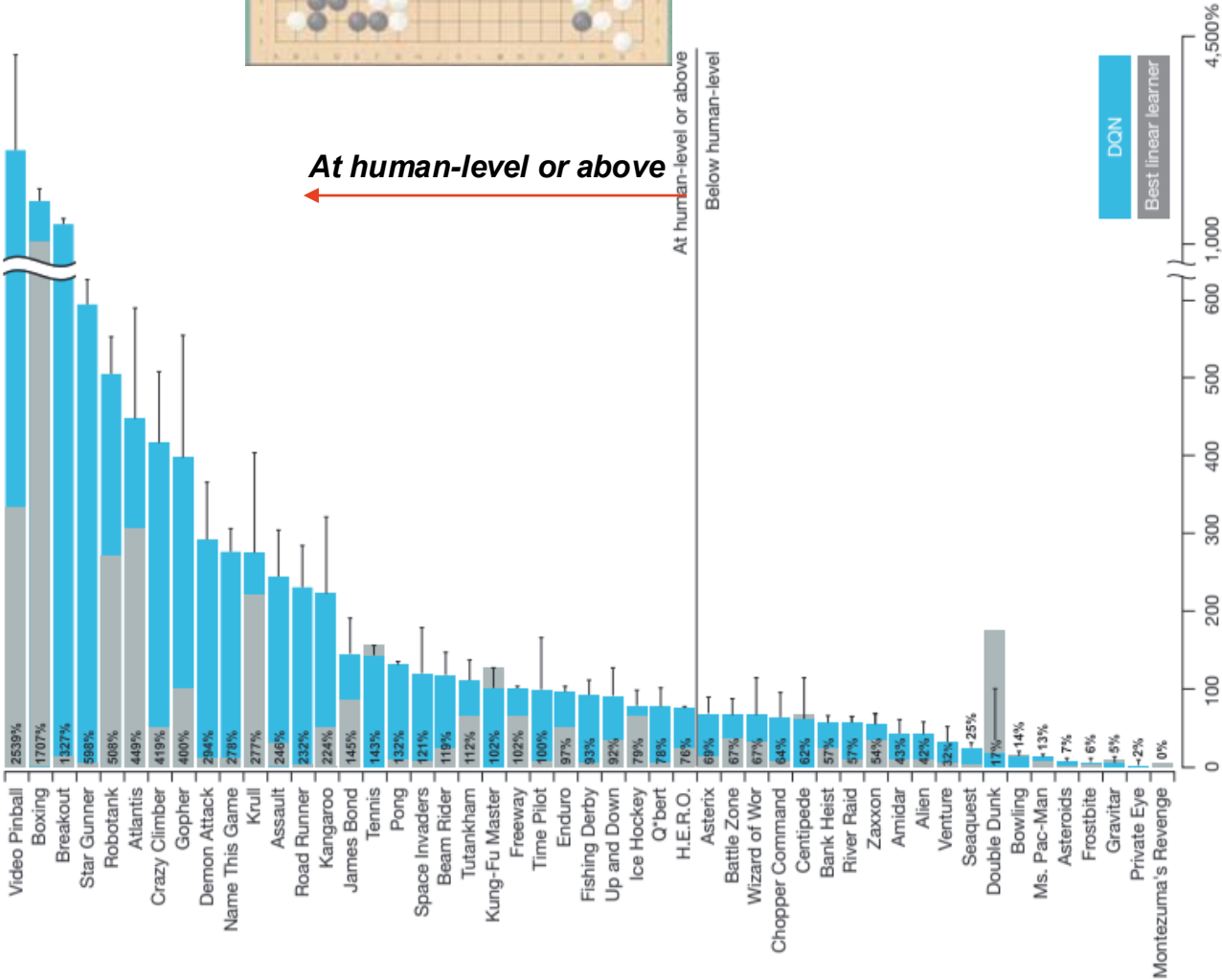
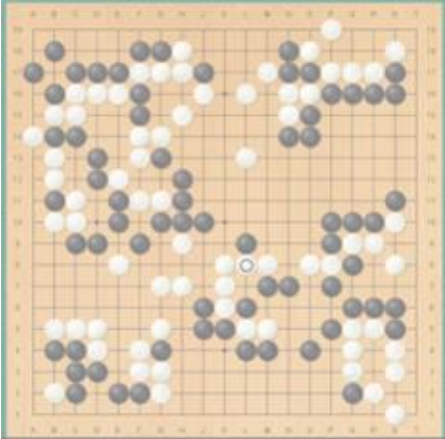
School of Computer Science



THE UNIVERSITY OF
SYDNEY

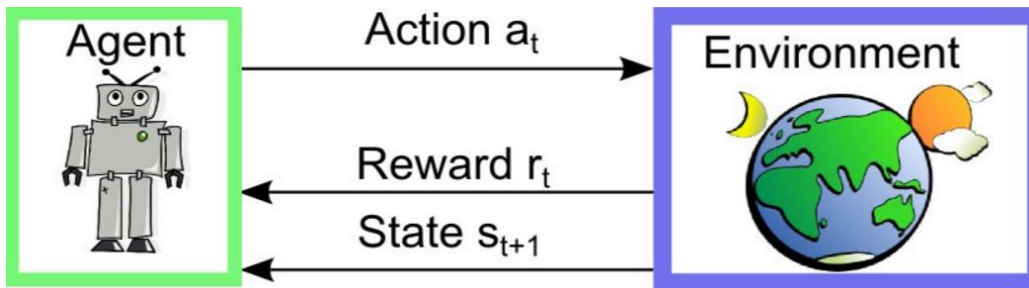


Deep RL Breakthroughs



Reinforcement Learning (RL) in a nutshell

- RL is a general-purpose framework for decision making
 - RL is for an **agent** to **act** with an **environment**
 - Each **action** influences the agent's future **state**
 - Feedback is given by a scalar **reward** signal
 - Goal: **select actions to maximize future reward**

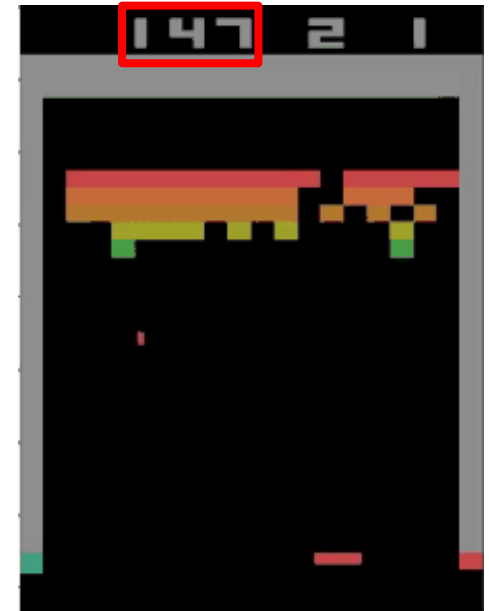
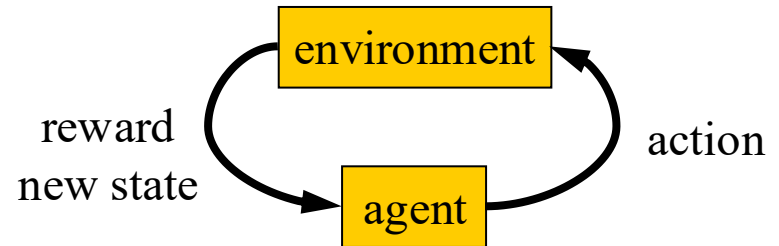


Actions: muscle contractions
Observations: sight, smell
Rewards: food

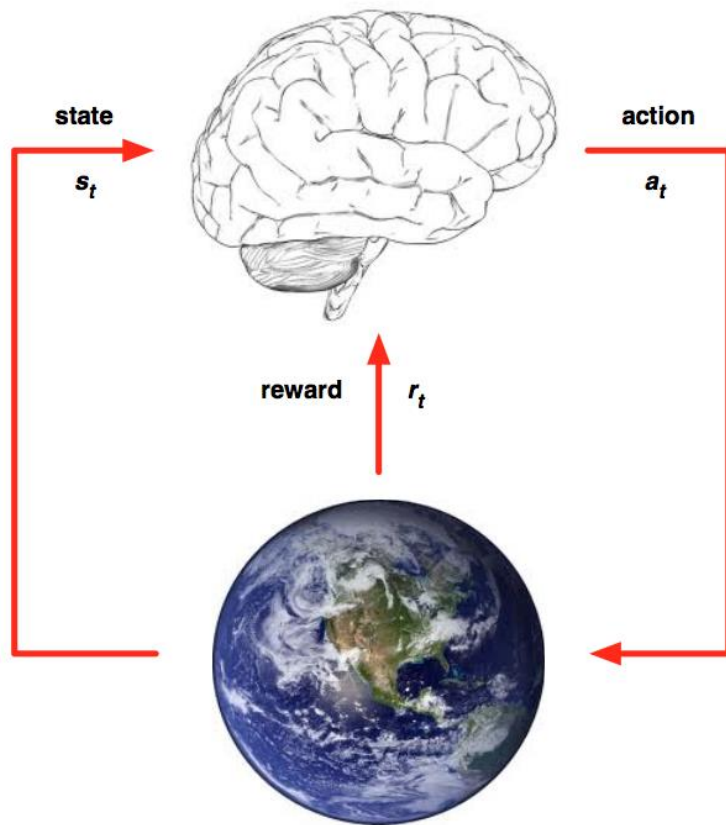
From the talk *Introduction to Deep Reinforcement Learning From Theory to Applications* by Siyi Li (HKUST)

Markov Decision Process (MDP)

- set of **states** S , set of **actions** A , initial state S_0
- transition model $P(s, a, s')$
 - $P(\text{frame}_{(t)}, \text{right}, \text{frame}_{(t')}) = 0.8$
- **reward** function $r(s)$
 - $r(\text{frame}_{(t)}) = +1$
- goal: maximize cumulative reward in the long run
- **policy**: mapping from S to A
 - $a = \pi(s)$ or $\pi(s, a)$ (deterministic vs. stochastic)
- reinforcement learning
 - transitions and rewards usually not available
 - how to change the policy based on experience
 - how to explore the environment



Markov Decision Processes (MDPs)



- ▶ At each step t the agent:
 - ▶ Receives state s_t
 - ▶ Receives scalar reward r_t
 - ▶ Executes action a_t
- ▶ The environment:
 - ▶ Receives action a_t
 - ▶ Emits state s_t
 - ▶ Emits scalar reward r_t

From the *Tutorial: Deep Reinforcement Learning* by David Silver, Google DeepMind

Computing return from rewards

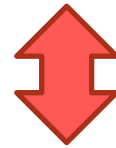
- episodic (vs. continuing) tasks
 - “game over” after N steps
 - optimal policy depends on N ; harder to analyze
- additive rewards
 - $V(s_0, s_1, \dots) = r(s_0) + r(s_1) + r(s_2) + \dots$
 - infinite value for continuing tasks
- discounted rewards
 - $V(s_0, s_1, \dots) = r(s_0) + \gamma * r(s_1) + \gamma^2 * r(s_2) + \dots$
 - value bounded if rewards bounded

The goal of RL is to find the policy which maximizes the expected return.

Value Function

- A value function is the prediction of the future return
- Two definitions exist for the value function
 - State value function
 - “How much reward will I get from state s ?”
 - expected return when starting in s and following π

$$V^\pi(s) = \mathbb{E} \left\{ \sum_{k=0}^{\infty} \gamma^k r_k \middle| s_0 = s, \pi \right\}$$



$$V^\pi(s) = \mathbb{E} \{ Q^\pi(s, a) \mid a \sim \pi(s, \cdot) \}$$

- State-action value function
- “How much reward will I get from action a in state s ?”
- expected return when starting in s , performing a , and following π

$$Q^\pi(s, a) = \mathbb{E} \left\{ \sum_{k=0}^{\infty} \gamma^k r_k \middle| s_0 = s, a_0 = a, \pi \right\}$$

Bellman Equation and Optimality

- Value functions decompose into **Bellman equations**, i.e. the value functions can be decomposed into *immediate reward plus discounted value of successor state*

$$V^\pi(s) = \mathbb{E} \left\{ \sum_{k=0}^{\infty} \gamma^k r_k \middle| s_0 = s, \pi \right\} \longrightarrow V^\pi(s) = \mathbb{E} \{ R(s, a, s') + \gamma V^\pi(s') \}$$

$$Q^\pi(s, a) = \mathbb{E} \left\{ \sum_{k=0}^{\infty} \gamma^k r_k \middle| s_0 = s, a_0 = a, \pi \right\} \begin{array}{c} \text{---} \\ \downarrow \end{array} Q^\pi(s, a) = \mathbb{E} \{ R(s, a, s') + \gamma Q^\pi(s', a') \}$$

- An **optimal** value function is the maximum achievable value.

$$V^*(s) = \max_{\pi} V^\pi(s) \quad Q^*(s, a) = \max_{\pi} Q^\pi(s, a)$$

Bellman Optimality Equation

- Optimality for value functions is governed by the **Bellman optimality equations**.
- Two equations:

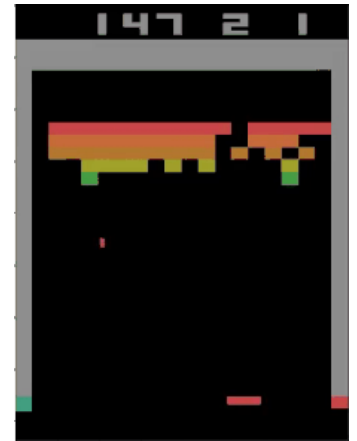
$$V^*(s) = \max_a \mathbb{E} \{ R(s, a, s') + \gamma V^*(s') \}$$

$$Q^*(s, a) = \mathbb{E} \left\{ R(s, a, s') + \max_{a'} \gamma Q^*(s', a') \right\}$$

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Q-Learning

Initializing the $Q(s, a)$ function



states

actions

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Noop	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Fire	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Right	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Left	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Q-Learning

$$Q^*(s, a) = \mathbb{E} \left\{ R(s, a, s') + \max_{a'} \gamma Q^*(s', a') \right\}$$

Initialize $Q(s, a)$ arbitrarily.

Start with s .

Before taking action a , we calculate the current expected return as

$$Q(s, a)$$

After taking action a , and observing r and s' , we calculate the target expected return as


$$R(s, a, s') + \max_{a'} \gamma Q(s', a')$$

$$\Delta Q(s, a) = R(s, a, s') + \max_{a'} \gamma Q(s', a') - Q(s, a)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \Delta Q(s, a)$$

Q-Learning

$$Q^*(s, a) = \mathbb{E} \left\{ R(s, a, s') + \max_{a'} \gamma Q^*(s', a') \right\}$$

Initialize $Q(s, a)$ arbitrarily.

Repeat (for each episode)

Initialize s

Repeat (for each step of the episode)

Choose a from s using a policy

Take action a , observe r, s'

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

$$s \leftarrow s'$$

Exploration and Exploitation

Initialize $Q(s, a)$ arbitrarily.

Repeat (for each episode)

Initialize s

Repeat (for each step of the episode)

Choose a from s using a policy

Take action a , observe r, s'

Update Q and s

Random policy; Exploration

Greedy policy; Exploitation

$$\pi(s) = \arg \max_a Q(s, a)$$

ϵ -greedy policy: With probability ϵ select a random action

Deep RL: Deep Learning + RL

- Traditional RL
 - low-dimensional state spaces
 - handcrafted features
- DL's representation power + RL's generalization ability
 - RL defines the objective
 - Deep Learning learns the representation

Deep Q-Learning

Deep Q-Learning



160

210

actions

states

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Noop	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Fire	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Right	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Left	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

$$\#states = 256^{210 \times 160}$$

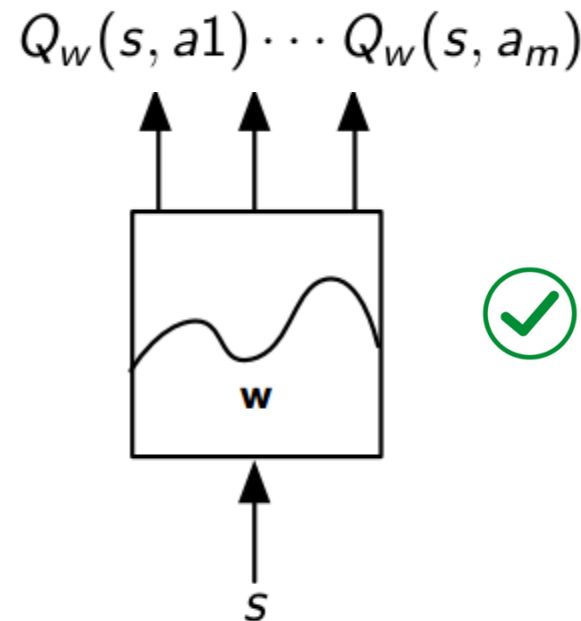
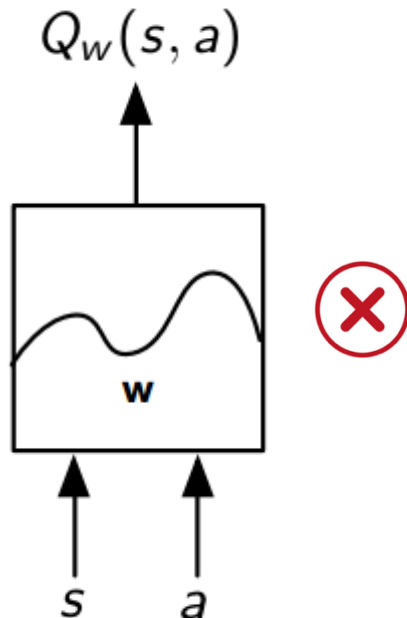
Value Function Approximation

$$Q(s, a) = f(s, a, w)$$

Q-Networks

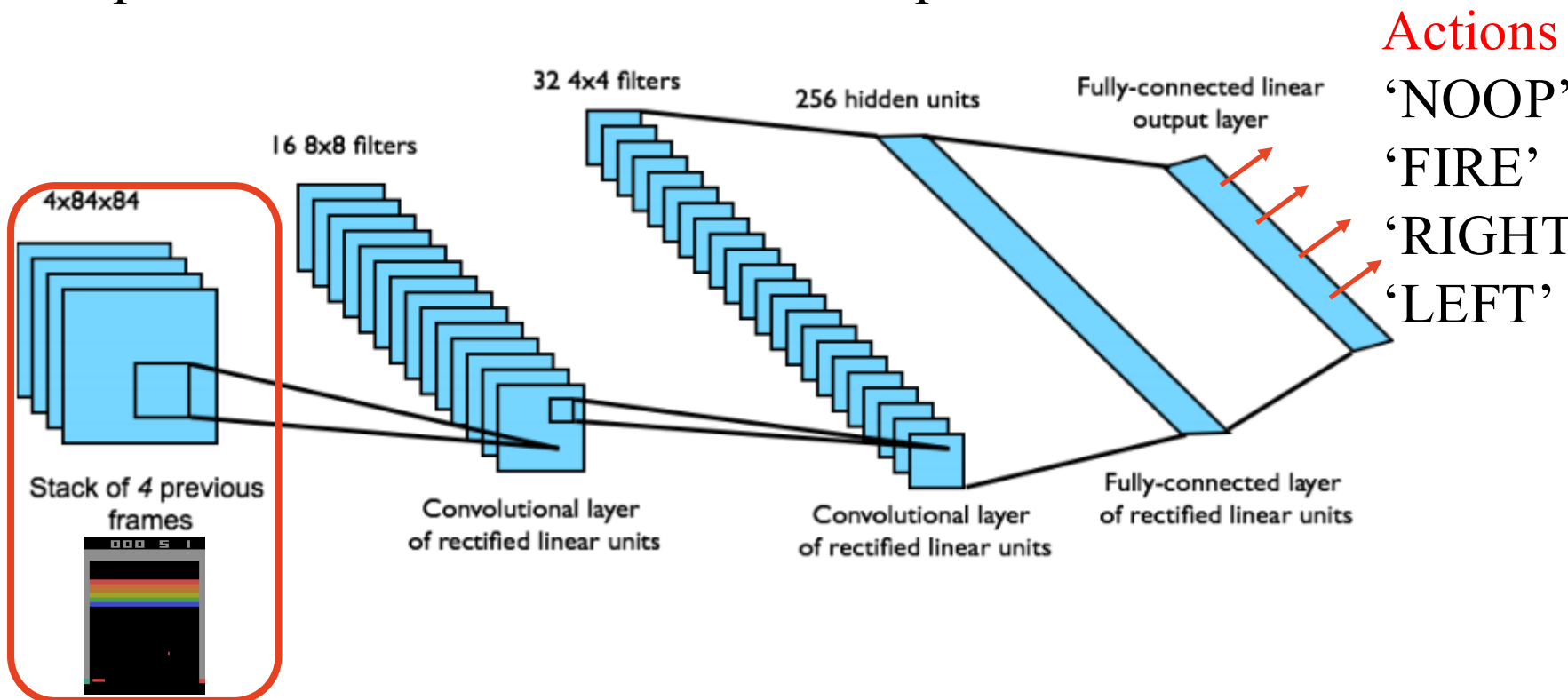
- Represent the **state-action value function** (discrete actions) by **Q-network** with weights w

$$Q_w(s, a) \approx Q^*(s, a)$$



Deep Q-Learning

- End-to-end learning of state-action values from raw pixels
- Input state is stack of raw pixels from last 4 frames
- Output are state-action values from all possible actions



From the *Tutorial: Deep Reinforcement Learning* by David Silver, Google DeepMind

Deep Q-Networks (DQN)

- Optimal Q-values obey Bellman equation

$$Q^*(s, a) = \mathbb{E}_{s'} \left\{ \underline{r + \gamma \max_{a'} Q(s', a')} | s, a \right\}$$

- Treat right-hand side as a target and minimize MSE loss by SGD

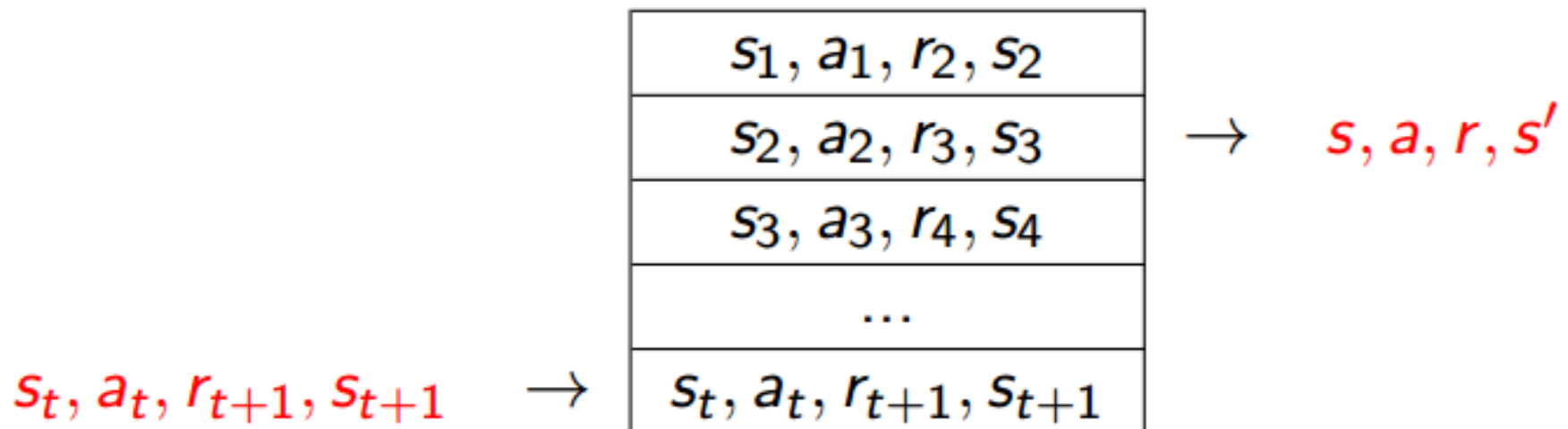
$$l = \left(\underline{r + \gamma \max_{a'} Q_w(s', a')} - Q_w(s, a) \right)^2$$

Target

- **Divergence issues** using neural networks due to
 - Correlations between samples
 - Non-stationary targets

Experience replay

- Build data set from agent's own experience
- Sample experiences uniformly from data set to remove correlations



Algorithm

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N

Initialize action-value function Q with random weights

for episode = 1, M **do**

 Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$

for $t = 1, T$ **do**

 With probability ϵ select a random action a_t ϵ -greedy
 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to

end for

end for

Reinforcement Learning in the Age of LLMs

- RLHF, DPO, SLiC-HF, and GRPO

From Game Playing to Language Alignment

RL for games

State = game screen

Action = joystick move

Reward = game score

RL to align LLMs

State = prompt + tokens so far

Action = next token

Reward = human preference

Core challenge:

No ground-truth reward function for open-ended language.

What makes a "good" answer? Helpful, harmless, honest — hard to formalize as a scalar.

Solution:

Learn rewards from human feedback — turn pairwise preferences ("A > B") into a training signal.

Key question: How do we turn pairwise human preferences into a training signal for LLMs?

Language Generation as an RL Problem

RL Concept		LLM Mapping
Environment		The token generation process
State	s_t	Prompt x + tokens so far $(x, y_1, \dots, y_{t-1})$
Action	a_t	Next token y_t sampled from vocabulary
Policy		$\pi_\theta(y_t \mid x, y_{<t})$ — LM's next-token distribution
Episode		One complete generation (prompt $\rightarrow$ EOS)
Reward		$r_\phi(x, y)$ — scalar score at end of episode

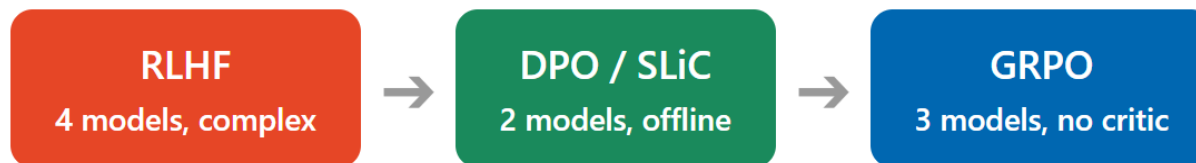
Each token generation step is an "action." A full response is an "episode." The reward is given only at the end — making credit assignment across tokens a key challenge.

Roadmap: Four Alignment Algorithms

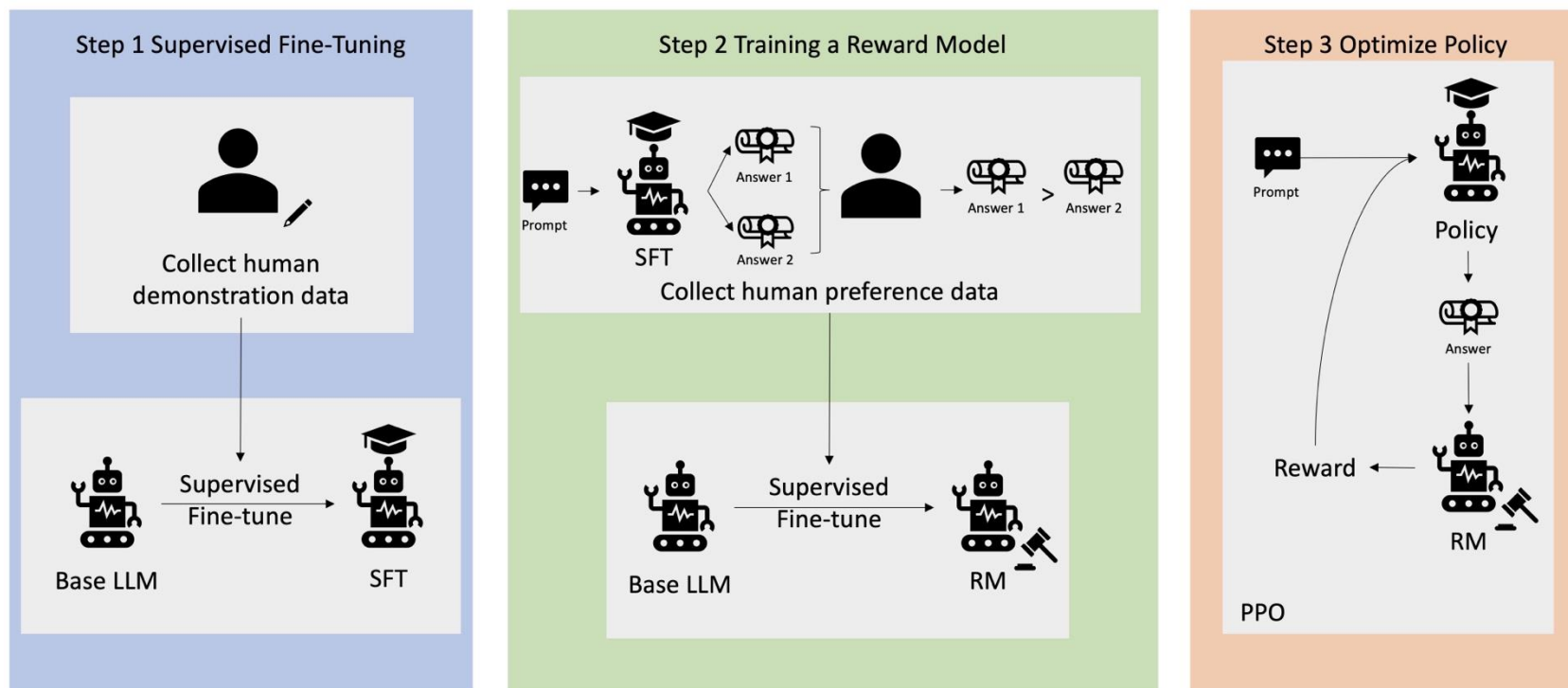
Method	Year	Key Idea	Reward Model?	RL Loop?	Critic?
RLHF (PPO)	2022	Full RL pipeline	Yes	Yes	Yes
DPO	2023	Closed-form, no RL	No	No	No
SLiC-HF	2023	Simple ranking loss	No	No	No
GRPO	2024	PPO without critic	Yes	Yes	No

Evolution: progressively simplifying the alignment pipeline

Each method = a different trade-off between theoretical rigor, compute cost, and simplicity.



The RLHF Pipeline: Three Stages



Stage 1 — SFT: Fine-tune pretrained LLM on human-written demonstrations

Stage 2 — Reward Model: Train scalar reward model from human preferences: "A > B"

Stage 3 — PPO: Optimize policy to maximize reward, constrained by KL penalty

Pipeline: InstructGPT (Ouyang et al., 2022); builds on Christiano et al. (2017), Stiennon et al. (2020).

Stage 1: Supervised Fine-Tuning (SFT)

- Start with a pretrained LM (e.g., GPT-3)
- Fine-tune on human-written demonstrations: (x, y) pairs, x = prompt, y = response
- Standard cross-entropy language modeling loss:

SFT LOSS

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{demo}}} \left[\sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t}) \right]$$

- Produces π^{SFT}
 - Initialization for reward model & policy
 - **Reference policy** for KL penalty in Stage 3

Intuition:

SFT teaches the format of good responses, but not the nuances of human preferences.

Stage 2: Reward Model Training

- Collect comparison data: humans indicate preferred vs. rejected
- Model preferences using the **Bradley-Terry model**:

BRADLEY-TERRY MODEL

$$P(y_w \succ y_l \mid x) = \sigma \left(\underbrace{r_\phi(x, y_w) - r_\phi(x, y_l)}_{\text{reward gap: should be positive}} \right)$$

- Train reward model $r_\phi(x, y)$ via maximum likelihood:

REWARD MODEL LOSS

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

- **Architecture**: SFT model with final layer replaced by linear projection to scalar output

Stage 3: RL Fine-Tuning — The RLHF Objective

- Optimize policy π_θ to maximize reward while staying close to reference π_{ref} :

RLHF OBJECTIVE

$$\max_{\theta} \mathbb{E}_{x, y \sim \pi_{\theta}} \left[\underbrace{r_{\phi}(x, y)}_{\text{preference signal}} - \underbrace{\beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})}_{\text{drift penalty}} \right]$$

Why the KL penalty?

- Prevents “reward hacking” — degenerate outputs exploiting the reward model
- Keeps generation quality close to SFT model
- β controls the trade-off

Per-token reward decomposition:

$$r_t = -\beta [\log \pi_{\theta}(y_t | \cdots) - \log \pi_{\text{ref}}(y_t | \cdots)]$$

(intermediate tokens: KL penalty only)

$$r_T = r_{\phi}(x, y) - \beta [\log \pi_{\theta}(y_T | \cdots) - \log \pi_{\text{ref}}(y_T | \cdots)]$$

(final token: reward model score + KL)

PPO: The Clipped Surrogate Objective

$$\min -\mathcal{L}_t^{CLIP}$$

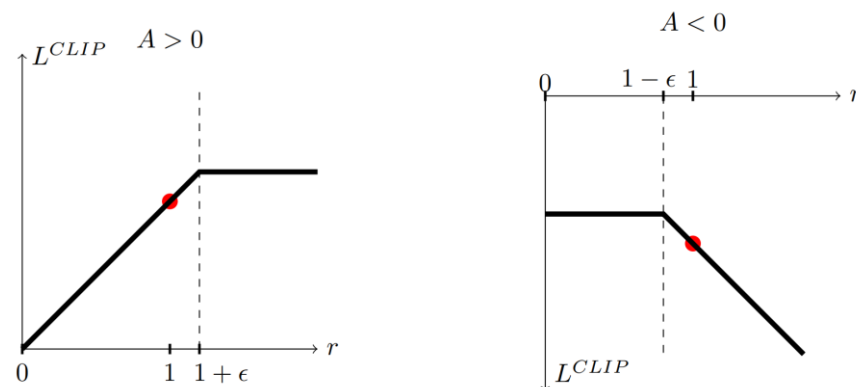
Importance sampling ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

The clipped surrogate objective:

PPO CLIPPED LOSS

$$\mathcal{L}_t^{CLIP} = \min\left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t\right)$$



PPO prevents the policy from **over-improving** actions (too large updates), even if the advantage suggests it.

ϵ : clip range, typically 0.1 – 0.2

Advantage (from Bellman equation):

$$\hat{A}_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

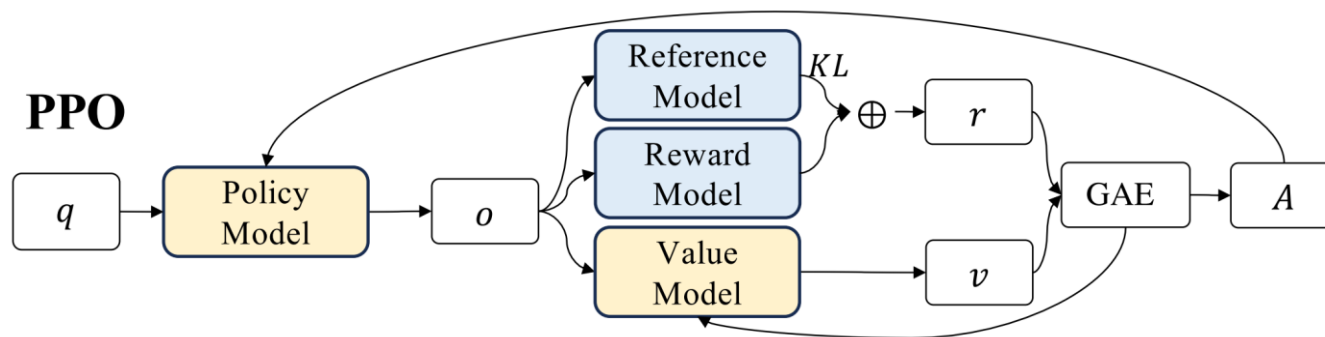
(reward + discounted future value – current value)

If $\hat{A}_t > 0$, $\max \rho_t$

If $\hat{A}_t < 0$, $\min \rho_t$

Left ($A > 0$): clip prevents making good actions too likely. **Right ($A < 0$):** clip prevents making bad actions too unlikely. Both prevent instability.

PPO for LLMs: Architecture Overview



Four models in GPU memory simultaneously:

Model	Role	Trainable?
π_{θ} Policy	The LLM being optimized	Yes
π_{ref} Reference	Frozen SFT copy — KL anchor	Frozen
r_{ϕ} Reward	Scores complete responses	Frozen
V_{ψ} Value	Critic for advantage estimation	Yes

Memory cost: For a 7B model → ~28B params total. With optimizer states: 100+ GB GPU memory. This is a major motivation for simpler alternatives.

InstructGPT: Key Results

1.3B InstructGPT preferred by humans over 175B GPT-3 (100 × smaller!)

Data used:

- ~13K demonstrations (SFT)
- ~33K comparisons (Reward Model)
- ~31K prompts (PPO training)

PPO-ptx variant:

$$\mathcal{L}(\theta) = \underbrace{\mathbb{E}[r_\phi - \beta \text{KL}]}_{\text{RLHF}} + \gamma \underbrace{\mathbb{E}[\log \pi_\theta(x)]}_{\text{pretrain mix}}$$

Improvements:

- Better truthfulness (TruthfulQA)
- Reduced toxic outputs
- Better instruction following

Remaining challenges:

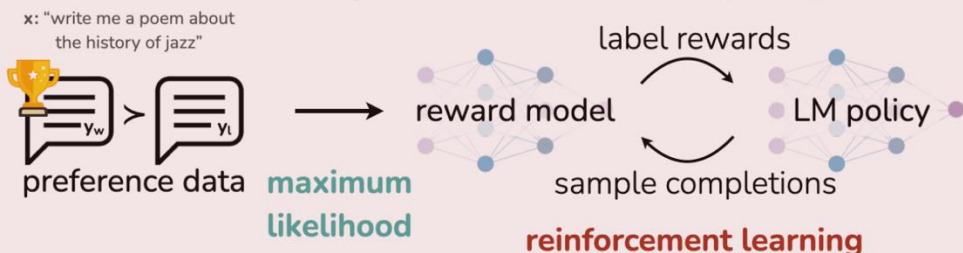
- Hallucination not fully solved
- Reward hacking still possible
- PPO training unstable & costly

InstructGPT established the standard RLHF recipe used by ChatGPT, Claude, Gemini, and others.

Direct Preference Optimization

DPO: Skip the Reward Model Entirely

Reinforcement Learning from Human Feedback (RLHF)



Direct Preference Optimization (DPO)



RLHF (PPO)

- 3 stages: SFT $\rightarrow$ RM $\rightarrow$ PPO
- 4 models in memory
- Online generation required
- Many hyperparameters
- Unstable training

DPO

- 2 stages: SFT $\rightarrow$ DPO fine-tuning
- 2 models only ($\pi_\theta, \pi_{\text{ref}}$)
- Offline — fixed preference dataset
- Mainly just β and learning rate
- Stable, supervised-style training

DPO: The Key Insight — Closed-Form Optimal Policy

Recall the RLHF objective: $\max_{\pi_{\theta}} \mathbb{E}[r(x, y)] - \beta \cdot D_{\text{KL}}[\pi_{\theta} || \pi_{\text{ref}}]$

- The KL-constrained RLHF objective has a closed-form optimal policy:

CLOSED-FORM OPTIMAL POLICY

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{r(x,y)}{\beta}\right)$$

- The reparameterization trick — solve for r in terms of the policy:

REWARD REPARAMETERIZATION

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \underbrace{\beta \log Z(x)}_{\text{depends only on } x}$$

The reward is implicitly defined by the policy — no need to learn it separately!

When comparing two responses for the same prompt, $Z(x)$ cancels — we never compute it.

DPO: From Preferences to Policy Loss

Substitute reparameterized reward into Bradley-Terry, $Z(x)$ cancels:

$$r(x, y_w) - r(x, y_l) = \beta \left[\log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right]$$

Replace π^* with π_θ and maximize likelihood of preference data:

DPO LOSS (MAIN RESULT)

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \left(\underbrace{\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}}_{\text{implicit reward of winner}} - \underbrace{\log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}}_{\text{implicit reward of loser}} \right) \right) \right]$$

A simple binary cross-entropy loss on preference pairs!

Theoretical guarantee: DPO provably optimizes the exact same KL-constrained objective as RLHF.

DPO: Gradient Intuition & Limitations

The gradient of the DPO loss:

DPO GRADIENT

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}} = -\beta \mathbb{E} \left[\underbrace{\sigma(\hat{u}_l - \hat{u}_w)}_{\text{large when model is wrong}} \left(\nabla_{\theta} \log \pi_{\theta}(y_w|x) - \nabla_{\theta} \log \pi_{\theta}(y_l|x) \right) \right]$$

Implicit hard-example mining:

- Weighting is large when model incorrectly ranks the pair
- Focuses learning on what model gets wrong
- Correctly-ranked pairs fade over time

Limitations:

- Offline distribution mismatch
- Sensitive to β
- No explicit reward for reuse
- Label noise sensitivity

SLiC-HF: An Even Simpler Alternative

- Direct contrastive/ranking loss on preference pairs — even simpler than DPO
- Pipeline: SFT → Calibration with ranking loss

MAX-MARGIN RANKING LOSS

$$\mathcal{L}_{\text{rank}} = \max(0, \delta - \log \pi_{\theta}(y_w|x) + \log \pi_{\theta}(y_l|x))$$

Hinge loss: push log-likelihood of preferred response at least higher than rejected

CROSS-ENTROPY CALIBRATION

$$\mathcal{L}_{\text{cal}} = -\log \sigma(\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x))$$

Same form as DPO but directly on log-likelihoods (no log-ratio with π_{ref})

REGULARIZATION + FULL OBJECTIVE

$$\mathcal{L}_{\text{reg}} = \max(0, \log \pi_{\text{ref}}(y_w|x) - \log \pi_{\theta}(y_w|x)) \quad \mathcal{L} = \mathcal{L}_{\text{rank}} + \lambda \mathcal{L}_{\text{reg}}$$

One-sided: activates only when π_{θ} drops the likelihood of preferred outputs below π_{ref}

Regularization activates only when policy drops preferred output likelihood below reference.

GRPO

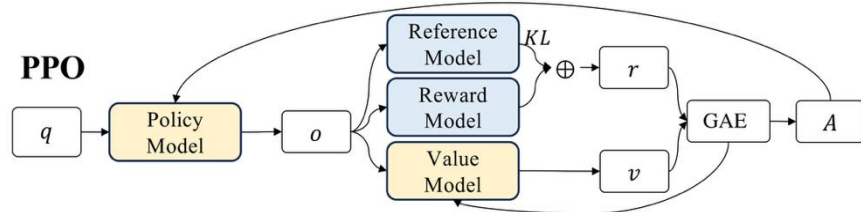
Group Relative Policy Optimization

- Shao et al., "DeepSeekMath" (2024) | DeepSeek-R1 (2025)

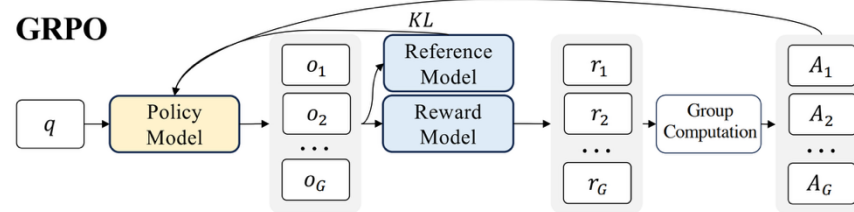
GRPO: Eliminating the Critic Network

Problem: PPO's value/critic is same size as policy — doubles memory.

GRPO's solution: Replace critic with group-level statistics. Zero extra parameters!



PPO: 4 models (requires Value Model)



GRPO: 3 models (Group Computation replaces Value)

Key: PPO uses learned value function for per-token advantages. GRPO uses **per-output advantages** from group reward statistics — zero extra parameters!

GRPO: Algorithm

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_{φ} ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

- 1: policy model $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$
- 2: **for** iteration = 1, ..., I **do**
- 3: reference model $\pi_{\text{ref}} \leftarrow \pi_{\theta}$
- 4: **for** step = 1, ..., M **do**
- 5: Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
- 6: Update the old policy model $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$
- 7: Sample G outputs $\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$ for each question $q \in \mathcal{D}_b$
- 8: Compute rewards $\{r_i\}_{i=1}^G$ for each sampled output o_i by running r_{φ}
- 9: Compute $\hat{A}_{i,t}$ for the t -th token of o_i through group relative advantage estimation.
- 10: **for** GRPO iteration = 1, ..., μ **do**
- 11: Update the policy model π_{θ} by maximizing the GRPO objective (Equation 21)
- 12: Update r_{φ} through continuous training using a replay mechanism.

Output π_{θ}

Step 1 — Group Sampling:

For each prompt x , sample G outputs ($G=16 - 64$)

Step 2 — Reward:

Score each output with reward model $r_i = R(x, y_i)$

Step 3 — Group-Relative Advantage:

GROUP-RELATIVE ADVANTAGE

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\})}$$

GRPO: The Objective Function

PPO-style clipping with group-relative advantages:

GRPO OBJECTIVE

$$J_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \left[\min\left(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon) \hat{A}_i\right) - \beta D_{\text{KL}} \right]$$

- $\rho_{i,t}(\theta) = \pi_{\theta}(y_{i,t}|x, y_{i,<t}) / \pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})$, per-token importance sampling ratio
- $\hat{A}_i$ is constant across all tokens (outcome-level advantage)

Key insight:

GRPO = REINFORCE with group-mean baseline + PPO clipping.

No bootstrapping, no value function approximation error — just simple statistics.

GRPO: Memory Efficiency & DeepSeek-R1

Component	PPO	GRPO
Policy model	Yes	Yes
Reference model	Yes	Yes
Reward model	Yes	Yes
Value/critic	Yes	No!
Total	4 models	3 models

DeepSeek-R1 (Jan 2025):

- GRPO on DeepSeek-V3 (671B MoE)
- **R1-Zero: GRPO + rule-based rewards only (no SFT!)**
 - Emergent chain-of-thought reasoning!
 - Self-verification, “aha moments”
- R1: cold-start SFT + GRPO + rejection sampling
- Critic at 671B = prohibitively expensive

Trade-off: GRPO samples G outputs per prompt (G=16–64), increasing inference cost. But cheaper than a full-size critic network.

R1-Zero showed that RL alone (with GRPO) can produce reasoning without any supervised examples!

Unified Comparison of All Methods

Aspect	RLHF (PPO)	DPO	SLiC-HF	GRPO
Pipeline	SFT→RM→PPO	SFT→DPO	SFT→Rank	SFT→Group PPO
Reward model	Trained	Implicit	Not needed	Trained/rule
Critic/Value	Yes (full)	No	No	No (group)
Models in mem	4	2	2	3
Online gen?	Yes	No	No	Yes
Loss	PPO clip	Log-ratio CE	Hinge/margin	Clip PG+group
Stability	Low	High	High	Mod–High
Notable users	ChatGPT	Llama 2/3	Research	DeepSeek-R1

Trend: Moving from complex RL pipelines toward simpler alternatives — while GRPO shows on-policy RL can be efficient for 671B+ models.

References

- Ouyang et al. (2022). "Training LMs to follow instructions with human feedback." NeurIPS.
- Schulman et al. (2017). "Proximal Policy Optimization Algorithms." arXiv:1707.06347.
- Christiano et al. (2017). "Deep RL from Human Preferences." NeurIPS.
- Stiennon et al. (2020). "Learning to Summarize with Human Feedback." NeurIPS.
- Rafailov et al. (2023). "Direct Preference Optimization." NeurIPS.
- Zhao et al. (2023). "SLiC-HF: Sequence Likelihood Calibration." arXiv:2305.10425.
- Yuan et al. (2023). "RRHF: Rank Responses to Align LMs." arXiv:2304.05302.
- Shao et al. (2024). "DeepSeekMath." arXiv:2402.03300.
- DeepSeek-AI (2025). "DeepSeek-R1." arXiv:2501.12948.
- Bradley & Terry (1952). "Rank Analysis of Incomplete Block Designs." Biometrika.

Thanks!